

Kubernetes Beginner Guide

Chapter 3: Kubernetes Networking

Explained in simple English, figure by figure. Theory only, no code.

What you will learn

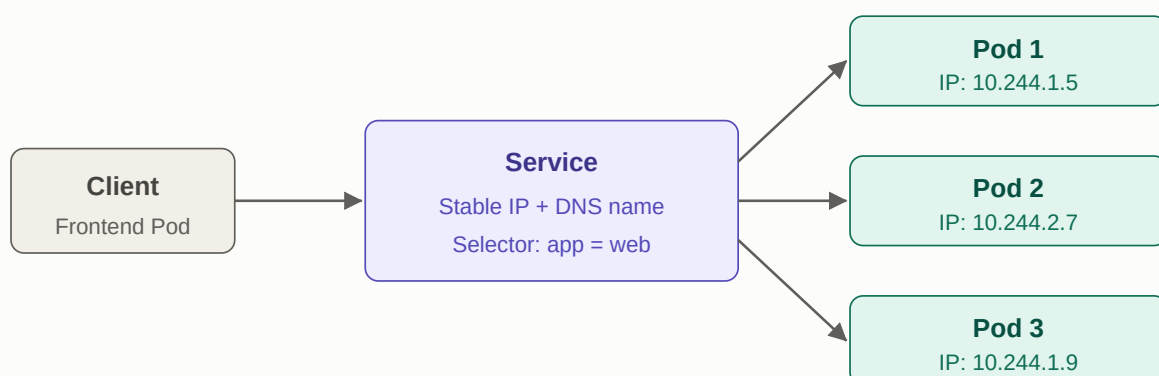
- What a **Service** is and how it finds and talks to Pods
- The three Service types: **ClusterIP**, **NodePort** and **LoadBalancer**
- What **Ingress** and an **Ingress Controller** are, and why we need both
- How Pods talk to each other, even on different worker nodes
- How **DNS** and service discovery let you use names instead of IPs
- What a **CNI plugin** is and why the cluster needs one

How to read this chapter

Every topic has one figure. Look at the figure first, then read the explanation under it. Each explanation walks through every part of the picture, gives the reason it exists, and ends with a real-life example. There are seven figures in total. This chapter answers two important interview questions: *How does a Service communicate with Pods?* and *How do containers communicate across different Worker Nodes?*

1. Service: a stable address for Pods

In Chapter 2 you learned that Pods die and come back with a new name and a new IP address. So if a frontend Pod remembers the IP of a backend Pod, that address stops working when the backend Pod is replaced. A **Service** solves this problem.



Pod IPs can change. The Service address never changes.

Figure 1. A Service gives a group of Pods one stable IP and name.

Explaining Figure 1

Client (left, gray). Any Pod that wants to talk to your app, for example a frontend that needs a backend. The client only knows one thing: the name or IP of the Service.

Service (middle, purple). A stable front door for a group of Pods. It has a **fixed virtual IP** (the ClusterIP) that does not change while the Service exists, a **DNS name** such as web-service, and a **selector** (here app = web). The selector is the label search from Chapter 2, Figure 5. The Service uses it to find its Pods.

Pods (right, teal). Pods 1, 2 and 3 all carry the label app = web, so the Service sends traffic to them. Each Pod has its own IP, and these IPs can change at any time.

How it works, step by step

- 1 The client sends a request to the Service address.
- 2 The Service picks one of the healthy Pods that match its selector.
- 3 The request goes to that Pod. The next request may go to a different Pod, so a Service is also a simple **load balancer**.

What happens when a Pod dies? The dead Pod is removed from the Service's list automatically. The replacement Pod has the same label, so it is added automatically. The client never notices, because it still uses the same Service address. Also, only Pods that are **ready** receive traffic, so a Pod that is still starting is skipped.

A detail for interviews. A Service is not a running program. It is a set of rules. The *kube-proxy* on each node (Chapter 1) writes these rules, so traffic sent to the Service IP is redirected to a real Pod IP.

Real-life example

A company has one main phone number. Employees may change desks or leave, but customers keep calling the same number, and the receptionist connects them to whoever is available.

2. Service types: ClusterIP, NodePort and LoadBalancer

A Service can be exposed in three main ways. The type you choose decides **who can reach it**.

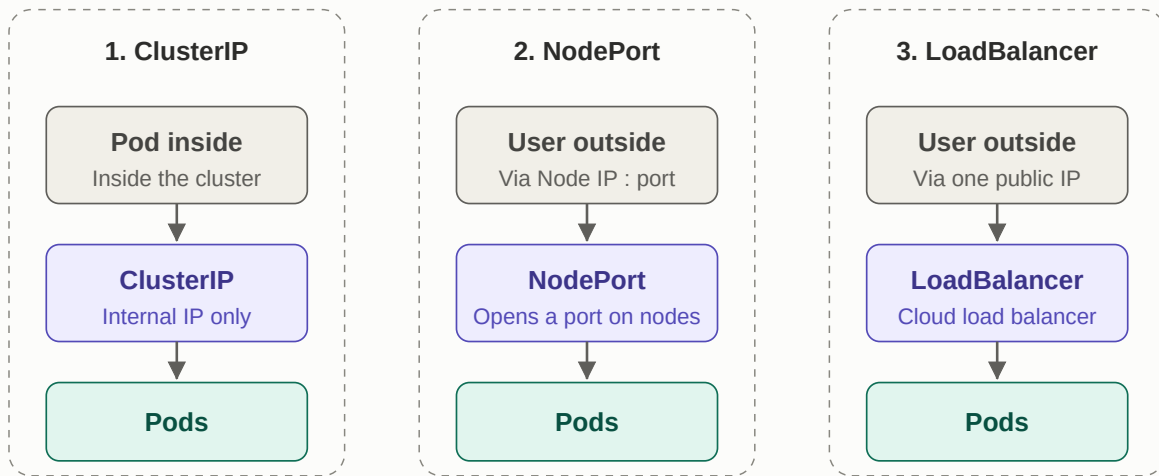


Figure 2. The three Service types differ in how traffic arrives. All end at the Pods.

Explaining Figure 2

Panel 1: ClusterIP (the default). The Service gets an internal IP that only works inside the cluster. Only other Pods can reach it, and nobody outside can. Use it for internal parts, such as a database or a backend API.

Panel 2: NodePort. Kubernetes opens the same port on every node. The port is between 30000 and 32767, for example 30080. An outside user can visit NodeIP:30080 and reach the Service from any node. Use it for quick tests, learning or simple setups. It is not nice for real users, because they must know a node IP and an odd port number, and if that node goes down, that address stops working.

Panel 3: LoadBalancer. Kubernetes asks your cloud provider (AWS, Google Cloud, Azure) to create a real load balancer with one public IP. Outside users connect to that IP, and the cloud load balancer sends traffic to your nodes and then to the Pods.

Caution

Each LoadBalancer Service usually creates a separate cloud load balancer, and each one costs money. With 20 apps you would have 20 load balancers. This is why Ingress (Figure 3) exists. On your own computer (minikube, kind) or on bare-metal servers there is no cloud to create one, so the Service may stay in the pending state.

The three types build on each other. A NodePort Service also creates a ClusterIP. A LoadBalancer Service also creates a NodePort and a ClusterIP. So each type includes the one before it.

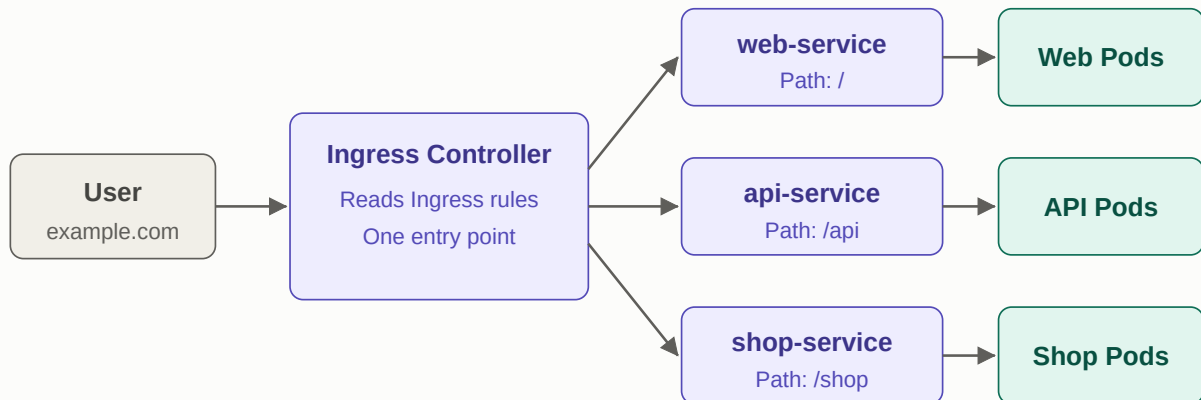
Type	Who can reach it	Typical use
ClusterIP	Only Pods inside the cluster	Databases, internal APIs
NodePort	Outside users, via Node IP and port	Testing, simple setups
LoadBalancer	Outside users, via one public IP	Public apps in the cloud

Real-life example

ClusterIP is an internal office extension that only works inside the building. NodePort is a side door on every floor that you must find by its number. LoadBalancer is the main entrance with a reception desk and a public address.

3. Ingress and Ingress Controller

With LoadBalancer Services, every app needs its own public IP and its own cloud load balancer. That is expensive and hard to manage. **Ingress** lets many apps share **one** entry point.



Rules match the host name and the path, then send traffic to the right Service.

Figure 3. The Ingress Controller reads Ingress rules and routes each request to the right Service.

Explaining Figure 3

User (left). A person types example.com/shop in a browser. The request arrives from the internet at the cluster's single entry point.

Ingress Controller (middle). This is the real worker. It is a program running in Pods inside your cluster, such as NGINX Ingress, Traefik or HAProxy. It receives all outside web traffic, reads the routing rules, and forwards each request to the correct Service.

Services and Pods (right). The path / goes to web-service and the Web Pods. The path /api goes to api-service and the API Pods. The path /shop goes to shop-service and the Shop Pods. The Ingress does not talk to Pods directly. It sends traffic to Services (usually ClusterIP), and the Services reach the Pods as in Figure 1.

Ingress and Ingress Controller are not the same thing

Name	What it is
Ingress	A Kubernetes object that holds the rules , for example "path /api goes to api-service"
Ingress Controller	The software that reads those rules and does the routing

An Ingress alone does nothing. Without an Ingress Controller installed, the rules are ignored.

Kubernetes does not install a controller by default, so you must install one. Managed clouds often offer

their own.

What Ingress can route by

- **Path:** example.com/api and example.com/shop go to different apps.
- **Host name:** api.example.com and shop.example.com go to different apps.

Why use Ingress?

- 1 **One entry point** and one public IP for many apps, so it is cheaper than many LoadBalancers.
- 2 **HTTPS (TLS) in one place.** You install the security certificate once, not in every app.
- 3 **Smart routing** by host and path, which a plain Service cannot do.

Two limits to know. Ingress is mainly for web traffic (HTTP and HTTPS). Also, the newer **Gateway API** is the modern successor with more features, but Ingress is still very common and is what freshers are asked about.

Real-life example

A large office building has one main gate with a reception desk. The receptionist reads a list, "Accounting is on floor 2, Sales is on floor 3", and directs each visitor. Without reception, every department would need its own separate entrance.

4. Pod-to-Pod communication

This section answers the question: **how do containers talk across different worker nodes?**

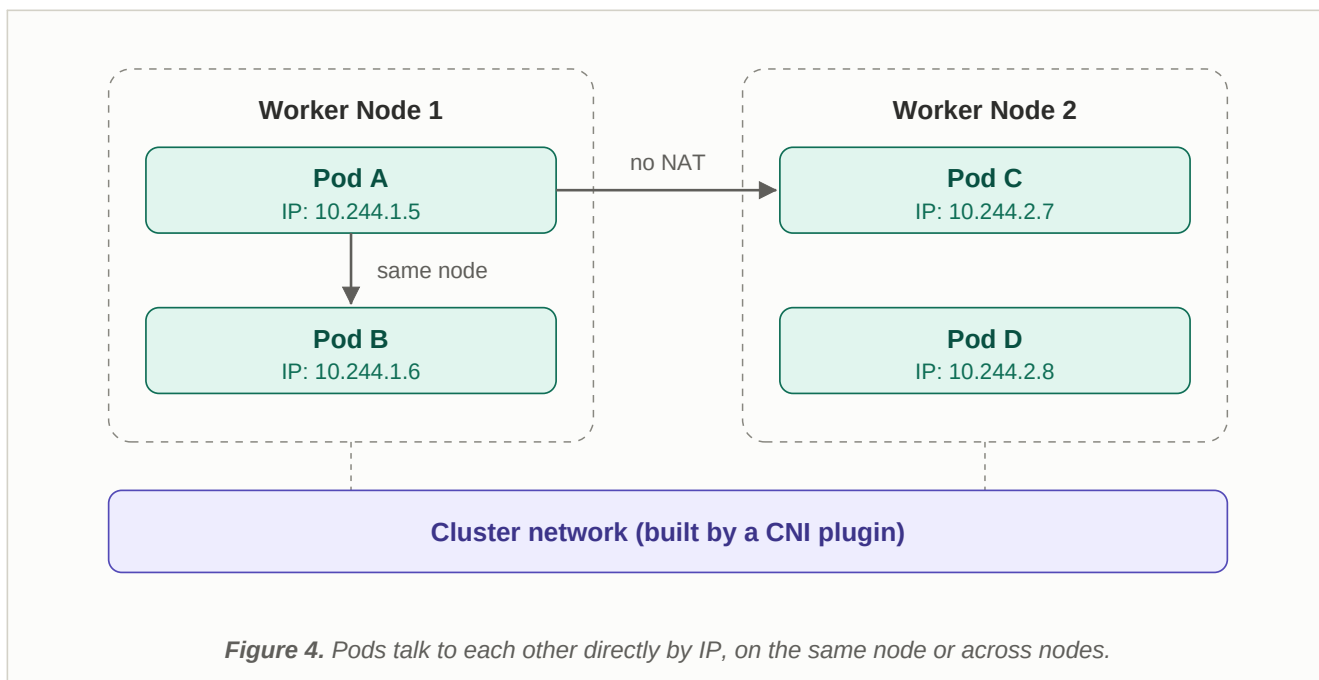


Figure 4. Pods talk to each other directly by IP, on the same node or across nodes.

Explaining Figure 4

The Kubernetes network rules. Kubernetes makes three promises:

- 1 Every Pod gets its **own IP address**.
- 2 Every Pod can talk to every other Pod **directly using that IP**, on any node, **without NAT** (address translation).

3 Nodes can talk to all Pods, and Pods can talk to nodes.

This is often called the **flat network**. To a Pod, the whole cluster looks like one big network, even though the Pods live on different machines.

Pod A to Pod B (same node). Both Pods are on Worker Node 1. Traffic stays inside that machine, through a virtual network bridge. This is fast, and it never leaves the node.

Pod A to Pod C (different nodes). Pod A uses Pod C's IP, 10.244.2.7, exactly as if it were next door. The cluster network carries the traffic from Node 1 to Node 2. Pod A does not need to know which node Pod C is on.

Who builds this network? The **CNI plugin** (bottom bar in the figure, and Figure 6). Kubernetes defines the rules, and a plugin does the actual work of connecting the nodes. Some plugins wrap traffic in a tunnel between nodes, and others use normal routing. As a fresher, you only need to know that the plugin makes it work.

Why each Pod has its own IP. In the old days, containers on one machine shared the machine's IP and had to fight over port numbers. In Kubernetes, two Pods can both use port 80 without conflict, because each has its own IP.

Important warning

Pod IPs are temporary (Figure 1). Pods *can* talk to each other by IP, but you should not do this in real apps. Use a Service instead.

Real-life example

A company has offices in two cities. Every employee has a direct phone extension. Calling a colleague in the other city works exactly like calling someone at the next desk, because the company phone system connects the offices behind the scenes.

5. DNS and service discovery

A Service has an IP, but IPs are hard to remember, and they can be different in every cluster. **Service discovery** means finding a Service by its **name** instead of its IP.

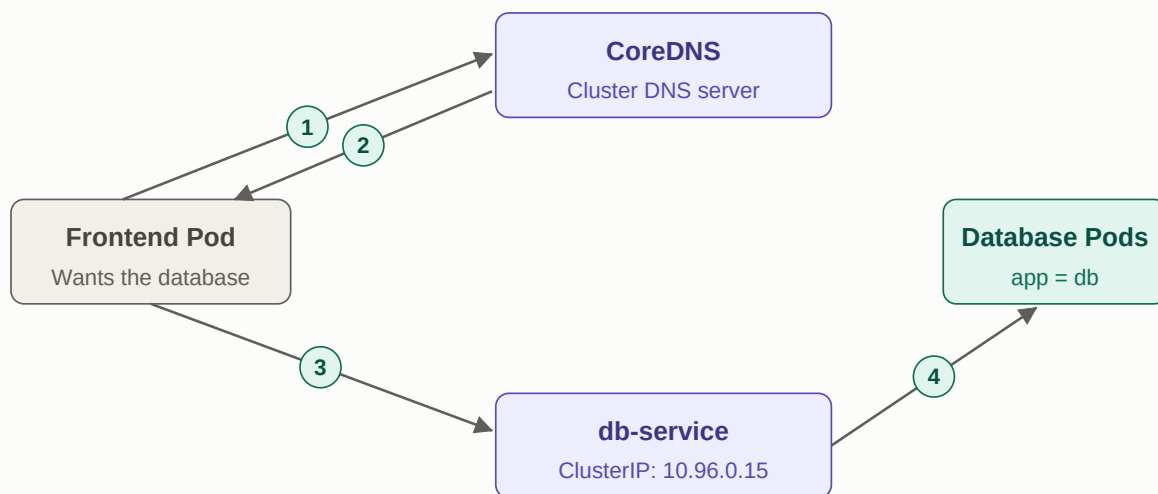


Figure 5. A Pod asks CoreDNS for a Service name, gets the IP, then connects to the Service.

Explaining Figure 5

Follow the numbered circles. This is what happens when a frontend Pod wants to reach the database.

Step 1: The Pod asks the DNS. The frontend Pod asks, "Where is db-service?" It sends this question to CoreDNS.

Step 2: CoreDNS answers. CoreDNS replies, "It is at 10.96.0.15." That is the Service's ClusterIP.

Step 3: The Pod connects to the Service. The frontend Pod now sends its request to 10.96.0.15.

Step 4: The Service forwards to a Pod. The Service uses its selector (`app = db`) and sends the request to one of the healthy database Pods, as in Figure 1.

What is CoreDNS?

- It is the cluster's built-in DNS server. It runs as Pods in the *kube-system* Namespace (Chapter 2).
- Every Pod is automatically configured to ask CoreDNS when it looks up a name.
- Each time you create a Service, CoreDNS automatically knows its name. You register nothing by hand.

The name of a Service has layers

Name you use	When to use it
db-service	The caller is in the same Namespace
db-service.prod	The caller is in a different Namespace , so you add the Namespace name
db-service.prod.svc.cluster.local	The full name , which always works

This is where Namespaces (Chapter 2, Figure 4) matter. The same name db-service can exist in dev and in prod, and the Namespace part tells DNS which one you mean.

Why is this useful? Your app config can say db-service and work in dev, test and prod without changing any IP. If the Pods behind the Service change or move, nothing breaks, because the name still points to the same Service.

Real-life example

You keep a phone contact called "Mom". You never remember her number, because you tap the name. If she changes her number, you update the contact and every call still works. CoreDNS is the phone contact book of the cluster.

6. CNI plugins

Kubernetes describes how Pod networking should behave (Figure 4), but it does **not** include the networking software itself. It hands this job to a **CNI plugin**. CNI means **Container Network Interface**, a standard way for Kubernetes to call a networking plugin.

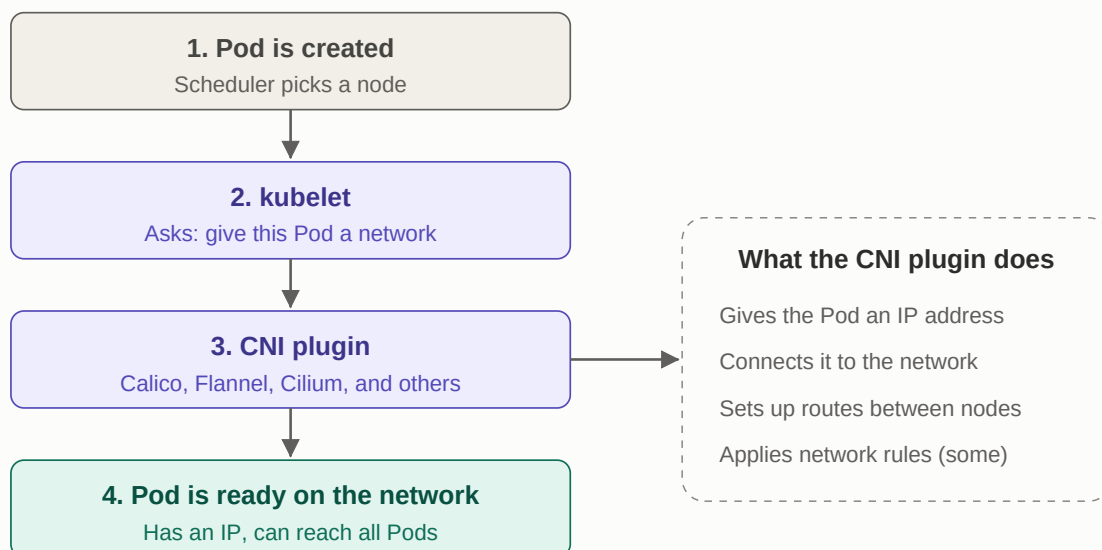


Figure 6. The kubelet calls the CNI plugin, which gives the Pod an IP and connects it to the network.

Explaining Figure 6

1 Pod is created. You create a Deployment, and the Scheduler picks a node for each Pod.

- 2 **kubelet.** The kubelet on that node starts the containers. It also needs a network for the Pod, so it calls the CNI plugin: "Please set up networking for this Pod."
- 3 **CNI plugin.** The plugin does the networking work (see the panel on the right of the figure).
- 4 **Pod is ready.** The Pod has an IP address and can talk to every other Pod, as in Figure 4.

What the plugin does: it gives the Pod an IP address from the cluster's address range, connects the Pod to the network on its node, and sets up routes so Pods on different nodes can reach each other. Some plugins also enforce Network Policies, which are firewall rules for Pods (Chapter 5).

Why is networking a plugin? Different companies have different needs: a small test cluster, a huge cloud cluster, or a high-security bank. A standard interface lets you choose the networking that fits, without changing Kubernetes itself.

Popular CNI plugins

Plugin	In short
Flannel	Very simple, good for learning and small clusters. It does not enforce Network Policies.
Calico	Very popular. Supports Network Policies and flexible routing.
Cilium	Modern and powerful, built on eBPF, with strong security and visibility features.
Cloud plugins	AWS VPC CNI and Azure CNI use the cloud's own network for Pods.

A very common problem

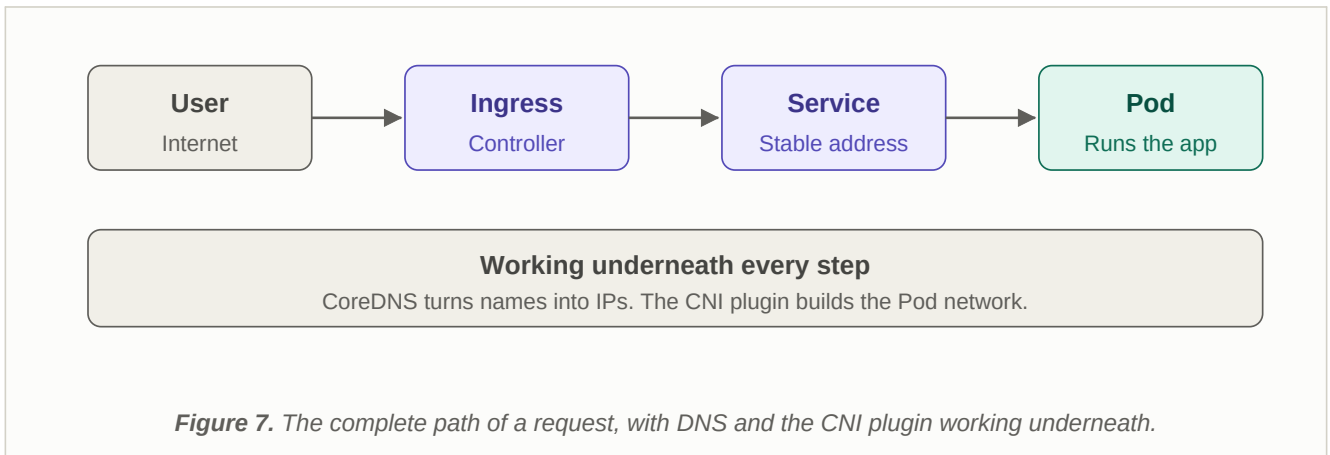
If no CNI plugin is installed, Pods cannot get IP addresses. Nodes may show NotReady, and Pods stay stuck in ContainerCreating. This is typical when someone builds a cluster by hand. Remember: no CNI plugin, no working cluster network.

Real-life example

Kubernetes is like a city that requires every house to have internet, but does not lay the cables itself. Each neighborhood hires an internet provider (the CNI plugin) that connects every house and gives it an address.

7. How everything fits together

Here is the full journey of a request, from the internet to your app.



Explaining Figure 7

- 1 The **user** visits `example.com/api`.
- 2 The **Ingress Controller** reads the rules and picks `api-service`.
- 3 The **Service** picks a healthy Pod using its selector.
- 4 If that Pod needs the database, it asks **CoreDNS** for `db-service` and connects to that Service.
- 5 All Pod-to-Pod traffic, even across nodes, works because the **CNI plugin** built the cluster network.

Chapter 3 summary

Figure	Main idea
1	A Service gives Pods one stable IP and name, and shares traffic among healthy Pods
2	ClusterIP is internal, NodePort opens a port on every node, LoadBalancer uses a cloud load balancer
3	Ingress rules plus an Ingress Controller give one entry point that routes by host and path
4	Every Pod has its own IP and can reach every other Pod directly, even across nodes
5	CoreDNS turns Service names into IPs, so you use names and not IPs
6	A CNI plugin (Calico, Flannel, Cilium) builds the Pod network; without one, Pods cannot get IPs
7	User to Ingress Controller to Service to Pod, with DNS and the CNI plugin underneath

Check yourself

Try to answer these in your own words, without looking at the notes:

- 1 Why do we need a Service if Pods already have IP addresses?
- 2 How does a Service know which Pods to send traffic to?
- 3 What is the difference between ClusterIP, NodePort and LoadBalancer? Which one is the default?
- 4 What is the difference between an Ingress and an Ingress Controller? What happens if there is no controller?
- 5 Why is Ingress cheaper than one LoadBalancer per app?
- 6 How does Pod A on Node 1 talk to Pod C on Node 2?
- 7 How does a Pod find db-service by name? Which component answers?
- 8 What does a CNI plugin do, and what can go wrong without one?

Next: Chapter 4, Storage

You will learn why containers need persistent storage, and how **Volumes**, **PersistentVolumes (PV)**, **PersistentVolumeClaims (PVC)** and **StorageClass** keep your data safe when Pods die.